SYSTEM README & ARCHITECTURAL REVIEW

1. System Overview and Architecture: Our system implements a "Scatter-Gather" disributed pattern using the following components, and ensured complete separation with the help of SQS and S3

Component Name	Technologies used	Primary Role
Local Application (the client)	Java	Runs on local computer, initiates the jobs, uploads input data to S3, launches&terminate the manager and waits for the final analyzation output
Manager	Java, EC2	Reads input files from S3, split the tasks, scales and launches the workers, tracks job completion using an in-memory state class
Workers	Java, EC2, Stanford Parser	Pulls single tasks, performs NLP analysis, uploads results to S3, informs the manager if tasks completions.
Communication Layer	SQS	Asynchronous task distribution and buffering traffic spikes
Storage Layer	S3	Persistent storage for input files, individual worker results, and the final HTML summary.

2. Architectural Principles & Security: we followed the principles of least privilege, and strictly avoid storing and sharing our credentials at all let alone in plain text:
   - EC2 Instances(Manager and Workers): Authintication is handled entirely by IAM rules, wich grant temporary permissions directly to the instance profile, allowing the code to use the AWS SDK to talk to SQS, S3, and EC2 without ever touching an Access Key, Secret Key or a Session Token.
   - Local Application: The Local App uses temporary Session Tokens retrieved from the lab environment. This is secure as the token is temporary and timed only to the current job.
   - Moving And Storing Data: All communication with S3 and SQS endpoints is secured using HTTPS as the default princible ensured by AWS.

3. Scalability: Our system is inherently scalable due to its reliance on AWS managed services:

    - Separating: The asynchronous nature of SQS means an increase of 1 million clients won't crash the Manager. The messages are simply buffered in the queue.
    - Elasticity (Scaling Out): The Manager monitors the load and uses `ec2.runInstances()`, to launch "hundreds" of Workers in parallel based on the load ratio and the available system capacity (the 18-worker cap).
    - Limit Handling: The system handles traffic spikes by letting SQS buffer the load and launches workers up to the defined hard limit (19 instances as defined by the rules of the lab), which is a crucial aspect of responsible cloud scalability management.

4. System Failure Handling: The system is designed to handle failure, particularly around the non-persistent nature of EC2 instances.

    - Worker Failure: All transient and internal Worker failures rely on relied on SQS Visibility Timeout**.** If a Worker stalls or dies while processing a task, it fails to send the completion signal `deleteMessage`. When the Visibility Timeout expires, the message automatically becomes Available again, and another Worker can pick it up for retry.
    - Data Loss: Solution: S3 Data Durability. All initial input data and all partial analysis results are stored on S3. If all Manager and Worker nodes die, the input and the partial output are still safe.

5. "Threads?": In our system, the use of multi-threading for core processing is generally not a good idea.

    - The system's primary task is Stanford CoreNLP parsing is deeply relied on CPU and memory. Utilizing threads on small EC2 cores. significantly degrades performance. The optimal strategy for both the Manager and Workers is to use a single thread. for the main processing loop, ensuring simplified state management and maximum computational efficiency.

6. Running Various Clients Simultaneously**:** The system is validated for concurrent client operation based on:

    - Unique Task IDs: Each job receives a unique UUID (Universally Unique ID).
    - State Separation: The Manager tracks each job separately in the Map.
    - No Interference: Workers use the shared SQS input queue but upload results to unique S3 keys which are extracted from the task_Id and the analyzation type, ensuring no client (and no sub-task) interferes with another's data.

7. Manager Sticking To Its Job: we insured that by preventing mixing of responsibilities:
    - Manager's Tasks (Mainly I/O): Tracks job states, Splits input, Launch workers, Gather and aggregate results.
    - Workers' Tasks (Heavy CPU&Memory): Download Text, Run Stanford Parser (the bottle neck), Upload Result.

    The Manager only performs light operations necessary for orchestration, leaving the massive CPU load of NLP analysis to the scaled-out Workers.

8. Balanced Workers-Work Load: this is insured because SQS acts as a centralized load balancer. Workers simply ask the queue for the next available task. The system ensures that:

    - No worker is sitting idle if there are messages in the queue.
    - The work is automatically distributed across all running instances without needing complex scheduling logic from the Manager.

9. Termination Process: The termination process is triggered by the optional terminate flag in the command line:

    - The Local App sends a final terminate message to the Manager.
    - The Manager completes the current job's aggregation.
    - The Manager calls `ec2.terminateInstances()` to explicitly kill all running Worker instances.
    - The Manager then terminates **itself** `ec2.terminateInstances(managerID)`.
    - The Local App exits `System.exit(0)`.

10. Distributed System: The system operates as a Distributed, Asynchronous System.

    - Distributed**:** The system spans three different physical locations/entities (Local machine, Manager EC2, Worker EC2s), communicating via the cloud's infrastructure (S3/SQS).
    - Asynchronous**:** The Manager does not sit and wait for a Worker to reply. It sends the message and moves on, waiting for the result to arrive later in a separate SQS message. Nothing is synchronously awaiting a response from another node (except the initial launch of the EC2 instance).